

CORRIDA

Documento Arquitetural do Sistema

Integrantes do grupo:

- José Anderson de Oliveira Silva
- Felipe Bezerra Ferreira de Lima
- Carlos Alberto Ferreira de Lima Neto
- Pedro Guilherme Soares de Lima
- Ryan Douglas Carvalho dos Santos

A página original do documento foi mantida integralmente.

CORRIDA

Documento Arquitetural do Sistema

Versão: 1.0

Data: 08/09/2026

Status: Em desenvolvimento

Equipe: 4 desenvolvedores e 1 Gerente de Projetos

1. Apresentação

Este documento apresenta a proposta arquitetural do sistema **Corrida**, uma plataforma destinada ao monitoramento e acompanhamento de corridas de rua.

O sistema permitirá que usuários criem uma conta, iniciem e acompanhem corridas, registrem sua localização por GPS, consultem históricos e estatísticas, descubram rotas e utilizem recursos básicos de segurança, como o cadastro de contatos de emergência.

O objetivo deste documento é registrar as principais decisões técnicas do projeto, estabelecendo uma base comum para o desenvolvimento do aplicativo mobile, da API e do banco de dados.

A arquitetura foi definida considerando o cenário inicial da empresa:

- equipe formada por quatro desenvolvedores e um gerente de projetos;
 - orçamento limitado;
 - prazo curto para o lançamento da primeira versão;
 - necessidade de manutenção simples;
 - expectativa de crescimento futuro do produto.
-

2. Problema que o sistema resolve

Corredores costumam utilizar diferentes aplicativos e ferramentas para controlar suas atividades, acompanhar o percurso, visualizar métricas e manter um histórico de desempenho.

O **Corrida** concentra essas funções em uma única solução, permitindo acompanhar a atividade esportiva de maneira simples, centralizada e segura.

A solução deverá oferecer uma experiência objetiva, com poucos passos para iniciar uma corrida e consultar as informações mais importantes, como tempo, distância, velocidade média e percurso realizado.

3. Requisitos arquiteturalmente relevantes

Os requisitos abaixo influenciam diretamente a escolha da arquitetura:

3.1 Funcionalidades principais

- Cadastro e autenticação de usuários;
- Login com e-mail e senha;
- Recuperação de senha;
- Criação, início, pausa, retomada e encerramento de corridas;
- Rastreamento da localização por GPS;
- Registro dos pontos do percurso;
- Consulta do histórico de corridas;
- Exibição de estatísticas de desempenho;
- Pesquisa e visualização de rotas;
- Compartilhamento de uma rota registrada;
- Cadastro de contato de emergência;
- Acionamento de alerta de segurança;
- Gerenciamento do perfil do usuário.

3.2 Requisitos não funcionais

- Comunicação entre aplicativo e servidor por HTTPS;
- Autenticação baseada em JWT;
- Controle de acesso aos dados do usuário;
- Validação dos dados recebidos pela API;
- Boa resposta das telas principais;
- Preservação dos dados de uma corrida em andamento;
- Facilidade de manutenção e evolução;
- Registro de logs e tratamento padronizado de erros;

- Possibilidade de crescimento sem reescrever todo o sistema.
-

4. Decisão arquitetural

Arquitetura escolhida: Monolito em Camadas

A primeira versão do **Corrida** será desenvolvida como um **monolito modular em camadas**, disponibilizado por meio de uma API REST.

Embora o sistema seja monolítico, seus módulos e responsabilidades serão separados internamente. Essa organização evita que o projeto se transforme em um código único e desorganizado, além de facilitar uma futura extração de módulos para microsserviços, caso o crescimento do produto justifique essa mudança.

A arquitetura lógica será composta por:

1. Cliente mobile;
 2. API REST;
 3. Middlewares de segurança e validação;
 4. Controllers;
 5. Services ou camada de negócio;
 6. Repositories ou camada de persistência;
 7. Banco de dados MySQL.
-

5. Justificativa da escolha

5.1 Custo

O monolito em camadas exige menos infraestrutura do que uma arquitetura baseada em microsserviços. Inicialmente, será necessário manter uma aplicação principal, um banco de dados e os serviços básicos de hospedagem.

Isso reduz custos com servidores, monitoramento, redes internas, filas, descoberta de serviços e ferramentas de observabilidade distribuída.

5.2 Velocidade de desenvolvimento

O prazo inicial é curto. Em um monolito, os desenvolvedores conseguem criar, testar e publicar os módulos em um único projeto, com menor quantidade de configurações e integrações entre serviços.

A comunicação interna ocorre por chamadas de métodos, e não por diversas requisições de rede entre serviços independentes. Essa característica acelera a construção do MVP.

5.3 Quantidade de desenvolvedores

A equipe possui quatro desenvolvedores. Para esse tamanho de equipe, a divisão em vários microsserviços poderia gerar mais trabalho operacional do que benefício técnico.

No monolito modular, os integrantes podem trabalhar em módulos diferentes, mantendo um padrão de código e uma implantação centralizada.

5.4 Facilidade de implantação

A aplicação poderá ser publicada como um único serviço, com configuração mais simples de ambiente, logs e monitoramento.

Essa característica é importante para o lançamento inicial, pois reduz a quantidade de pontos que podem apresentar falhas durante a implantação.

5.5 Escalabilidade

O monolito atende à escalabilidade inicial do sistema. A aplicação poderá ser executada em mais de uma instância quando necessário, utilizando um balanceador de carga e um banco de dados adequadamente configurado.

No futuro, módulos que apresentarem maior demanda, como GPS, estatísticas ou rotas, poderão ser separados gradualmente em serviços independentes.

5.6 Complexidade da infraestrutura

Microsserviços normalmente exigem gateway, comunicação entre serviços, múltiplos bancos ou contextos de dados, observabilidade distribuída, gerenciamento de versões e estratégias de tolerância a falhas.

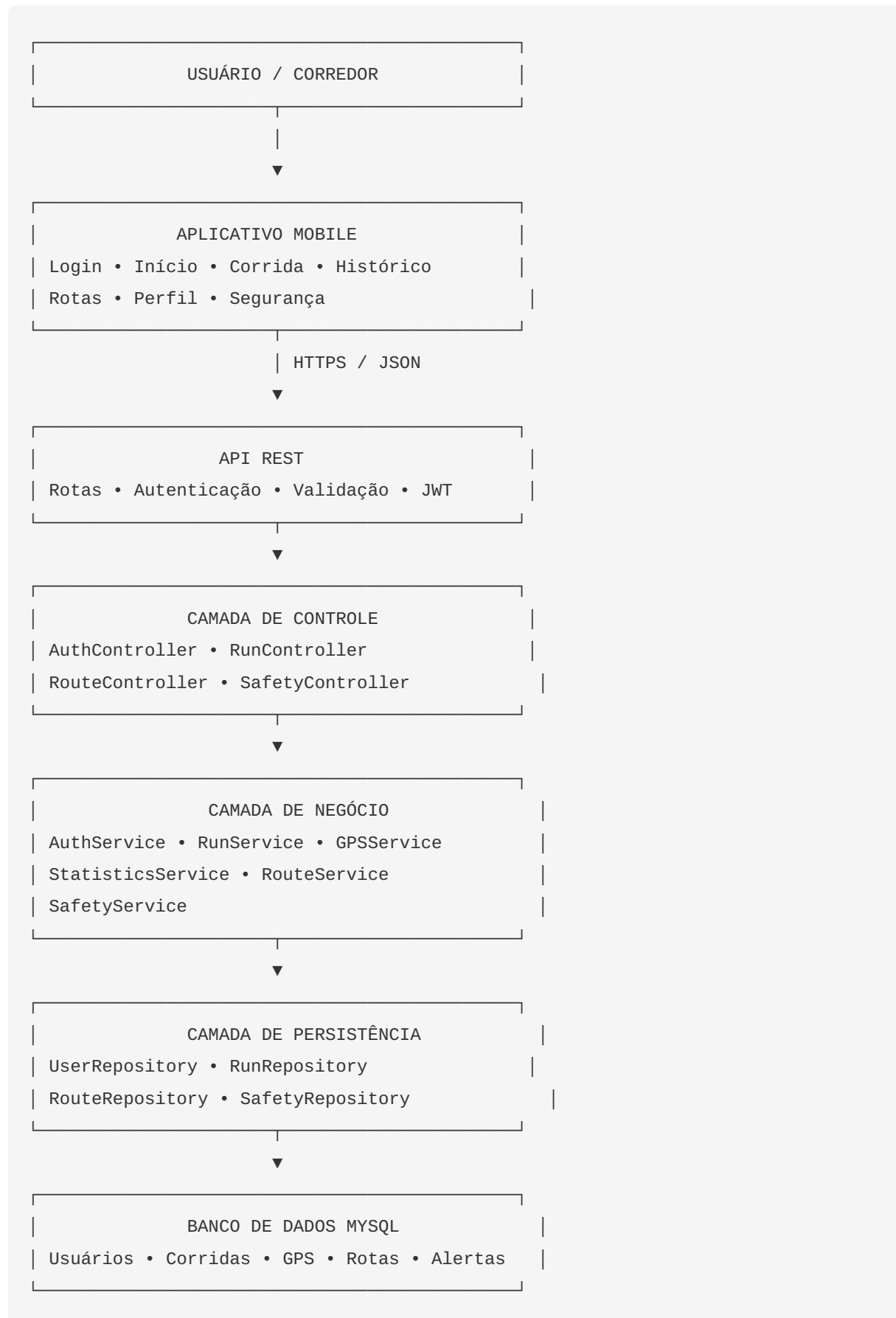
Para a primeira versão do **Corrida**, essa complexidade não é justificável diante do orçamento e do tamanho da equipe.

5.7 Manutenção futura

A manutenção será facilitada pela separação interna entre apresentação, controle, negócio e persistência.

A arquitetura não permite que a interface acesse diretamente o banco de dados. As regras ficam concentradas nos serviços, e o acesso aos dados é realizado pelos repositórios. Com isso, alterações em uma camada tendem a causar menos impacto nas demais.

6. Visão geral da arquitetura



7. Responsabilidades das camadas

7.1 Camada de apresentação

É representada pelo aplicativo mobile e pelas telas utilizadas pelo corredor.

Responsabilidades:

- apresentar informações ao usuário;
- coletar comandos e dados de entrada;
- solicitar permissões de localização;
- exibir o estado atual da corrida;
- mostrar mensagens de sucesso e erro;
- consumir a API REST.

Exemplos de telas:

- Login;
- Cadastro;
- Tela inicial;
- Controle da corrida;
- Mapa e rastreamento GPS;
- Histórico;
- Rotas;
- Perfil;
- Segurança do corredor.

A camada de apresentação não deverá implementar regras centrais do negócio nem acessar diretamente o banco de dados.

7.2 Camada de controle

Recebe as requisições HTTP, interpreta os parâmetros, chama os serviços adequados e devolve as respostas para o aplicativo.

Exemplos:

- `AuthController` ;
- `UserController` ;
- `RunController` ;
- `RouteController` ;

- `StatisticsController` ;
- `SafetyController` .

Os controllers devem permanecer objetivos. Eles não devem concentrar cálculos complexos ou regras extensas.

7.3 Camada de negócio

Concentra as regras e decisões do sistema.

Exemplos de responsabilidades:

- validar transições de estado de uma corrida;
- impedir que um usuário altere a corrida de outra pessoa;
- calcular duração e distância;
- consolidar estatísticas;
- validar a existência de um contato de emergência;
- criar tokens de compartilhamento;
- aplicar regras de autenticação e autorização.

Exemplos de serviços:

- `AuthService` ;
- `UserService` ;
- `RunService` ;
- `GpsService` ;
- `StatisticsService` ;
- `RouteService` ;
- `SafetyService` .

7.4 Camada de persistência

Responsável pela comunicação com o MySQL por meio do Prisma ORM.

Exemplos:

- `UserRepository` ;
- `RunRepository` ;
- `RunPointRepository` ;
- `RouteRepository` ;

- `EmergencyContactRepository` ;
- `SafetyAlertRepository` .

Essa camada deve esconder os detalhes de acesso ao banco de dados das demais camadas.

8. Módulos principais

A aplicação será organizada nos seguintes módulos:

Autenticação e usuários

Responsável por cadastro, login, recuperação de senha, sessão, perfil e autorização.

Corridas

Responsável pelo ciclo de vida da corrida: pronta, em execução, pausada, finalizada ou cancelada.

GPS

Responsável pelo recebimento e armazenamento dos pontos de localização associados a uma corrida.

Histórico e estatísticas

Responsável por consultar corridas concluídas e calcular informações consolidadas de desempenho.

Rotas

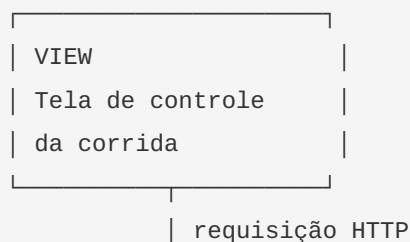
Responsável pela pesquisa, consulta e apresentação de rotas disponíveis.

Segurança

Responsável pelos contatos de emergência e pelos alertas acionados pelo usuário.

9. Aplicação do padrão MVC

O padrão MVC será aplicado principalmente nos módulos que possuem interação direta com o usuário. Como exemplo, será utilizado o módulo de **Corridas**.





View

No aplicativo mobile, a View corresponde às telas de controle da corrida. Ela apresenta o tempo, a distância, o status, o mapa e os botões de iniciar, pausar, retomar e encerrar.

Controller

O `RunController` recebe as chamadas da API, como a criação de uma corrida ou a atualização do seu status. Ele valida a estrutura básica dos dados e encaminha a operação para o serviço.

Model

O Model representa a corrida e as regras relacionadas ao seu comportamento. O `RunService` controla as regras de negócio, enquanto as entidades e os tipos representam os dados manipulados.

Exemplos de regras:

- uma corrida pronta pode ser iniciada;
 - uma corrida em execução pode ser pausada ou encerrada;
 - uma corrida pausada pode ser retomada ou encerrada;
 - somente o proprietário pode modificar a corrida;
 - uma corrida só aparece no histórico depois de finalizada.
-

10. Ciclo de vida da corrida

```
READY — iniciar —> RUNNING
RUNNING — pausar —> PAUSED
PAUSED — retomar —> RUNNING
RUNNING — encerrar —> FINISHED
PAUSED — encerrar —> FINISHED
RUNNING — cancelar —> CANCELLED
PAUSED — cancelar —> CANCELLED
```

A camada de negócio será responsável por impedir transições inválidas. Por exemplo, uma corrida finalizada não poderá voltar ao estado de execução.

11. API REST

A API utilizará o prefixo `/api/v1` e respostas em JSON. Todas as rotas privadas exigirão autenticação por Bearer Token, exceto as rotas públicas de cadastro e login.

Método	Endpoint	Finalidade
POST	<code>/api/v1/auth/register</code>	Criar uma conta
POST	<code>/api/v1/auth/login</code>	Autenticar o usuário
POST	<code>/api/v1/auth/forgot-password</code>	Solicitar recuperação de senha
GET	<code>/api/v1/users/me</code>	Consultar perfil do usuário autenticado
PATCH	<code>/api/v1/users/me</code>	Atualizar perfil
POST	<code>/api/v1/runs</code>	Criar ou iniciar uma corrida
PATCH	<code>/api/v1/runs/:id</code>	Pausar, retomar, encerrar ou cancelar uma corrida
GET	<code>/api/v1/runs/:id</code>	Consultar uma corrida
POST	<code>/api/v1/runs/:id/points</code>	Enviar pontos GPS
GET	<code>/api/v1/runs/:id/points</code>	Consultar o percurso GPS
GET	<code>/api/v1/runs/history</code>	Consultar histórico de corridas
GET	<code>/api/v1/statistics/summary</code>	Consultar resumo de estatísticas

Método	Endpoint	Finalidade
GET	/api/v1/routes	Listar e pesquisar rotas
GET	/api/v1/routes/:id	Consultar detalhes de uma rota
POST	/api/v1/runs/:id/share	Gerar compartilhamento de rota
GET	/api/v1/safety/emergency-contacts	Listar contatos de emergência
POST	/api/v1/safety/emergency-contacts	Cadastrar contato de emergência
POST	/api/v1/safety/alerts	Registrar alerta de segurança

Exemplo: iniciar uma corrida

Requisição

```
POST /api/v1/runs
Authorization: Bearer <token>
Content-Type: application/json
```

```
{
  "startedAt": "2026-09-08T08:00:00Z"
}
```

Resposta esperada

```
{
  "id": 101,
  "userId": 7,
  "status": "RUNNING",
  "startedAt": "2026-09-08T08:00:00Z",
  "distanceKm": 0,
  "durationSeconds": 0
}
```

Exemplo: atualizar o estado da corrida

Requisição

```
PATCH /api/v1/runs/101
Authorization: Bearer <token>
Content-Type: application/json
```

```
{
  "action": "PAUSE"
}
```

Resposta esperada

```
{
  "id": 101,
  "status": "PAUSED",
  "distanceKm": 2.35,
  "durationSeconds": 845
}
```

12. Persistência de dados

O banco de dados escolhido é o MySQL 8 ou versão superior, acessado por meio do Prisma ORM.

Principais tabelas:

Tabela	Finalidade
users	Dados da conta e do perfil
user_settings	Preferências do usuário
runs	Informações principais das corridas
run_points	Pontos de localização GPS
routes	Rotas disponíveis para pesquisa
route_points	Pontos que formam uma rota
shared_routes	Compartilhamentos de percursos
emergency_contacts	Contatos de emergência
safety_alerts	Alertas de segurança

Tabela	Finalidade
refresh_tokens	Controle de sessões
password_resets	Recuperação de senha
audit_logs	Registro de ações relevantes

A tabela `run_points` será separada da tabela `runs`, pois uma única corrida pode possuir centenas ou milhares de pontos GPS.

Relacionamentos principais:

- um usuário possui várias corridas;
- uma corrida possui vários pontos GPS;
- um usuário pode possuir vários contatos de emergência;
- uma rota possui vários pontos;
- uma corrida pode possuir compartilhamentos;
- um usuário pode registrar vários alertas de segurança.

13. Segurança arquitetural

A arquitetura deverá aplicar as seguintes medidas:

- senhas armazenadas com hash bcrypt;
- JWT com tempo de expiração reduzido;
- refresh tokens revogáveis;
- autorização baseada no proprietário do recurso;
- validação de dados com Zod;
- rate limiting em login e operações sensíveis;
- uso de HTTPS em produção;
- configuração de Helmet e CORS;
- não exposição de senhas, tokens ou segredos nas respostas;
- armazenamento cuidadoso de dados de localização;
- variáveis sensíveis mantidas em arquivo de ambiente;
- logs de ações importantes para auditoria.

A localização GPS será tratada como dado sensível. O sistema deverá coletá-la somente quando necessário e conforme a autorização concedida pelo usuário.

14. Organização do projeto

A estrutura escolhida será organizada por módulos, mantendo a separação interna por responsabilidade.

```
backend/  
├─ src/  
│   ├─ config/  
│   ├─ routes/  
│   ├─ controllers/  
│   ├─ services/  
│   ├─ repositories/  
│   ├─ middlewares/  
│   ├─ validators/  
│   ├─ utils/  
│   └─ modules/  
│       ├─ auth/  
│       ├─ users/  
│       ├─ runs/  
│       ├─ gps/  
│       ├─ routes/  
│       ├─ history/  
│       └─ statistics/  
│           └─ safety/  
│   └─ app.js  
└─ server.js  
├─ prisma/  
│   ├─ schema.prisma  
│   └─ migrations/  
│       └─ seed.js  
├─ tests/  
├─ .env.example  
├─ package.json  
└─ README.md
```

A organização por módulos permite que cada área do sistema evolua de forma mais independente, sem abandonar a simplicidade de implantação do monolito.

15. Tecnologias previstas

Tecnologia	Utilização
Node.js	Execução do backend
Express	Servidor HTTP e API REST
Prisma	ORM, migrations e acesso ao banco
MySQL 8+	Persistência dos dados
JWT	Autenticação
bcrypt	Hash de senhas
Zod	Validação de dados
Swagger/OpenAPI	Documentação da API
Pino	Logs estruturados
Jest ou Vitest	Testes unitários
Supertest	Testes de integração
dotenv	Configuração por ambiente

16. Implantação inicial

A primeira implantação poderá utilizar a seguinte configuração:



O ambiente deverá possuir configurações separadas para desenvolvimento, testes e produção. As credenciais do banco e as chaves de autenticação não deverão ser armazenadas no repositório de código.

A aplicação poderá ser empacotada em um container posteriormente, mas a primeira entrega deve priorizar simplicidade e estabilidade.

17. Estratégia de desenvolvimento

A implementação deverá seguir uma ordem que reduza riscos e permita validar rapidamente o produto:

1. Configuração do projeto Node.js, Express, Prisma e MySQL;
2. Criação do modelo de dados e migrations;
3. Cadastro, login e autenticação JWT;
4. Gerenciamento do perfil;
5. Criação e controle do ciclo de vida da corrida;
6. Recebimento e armazenamento de pontos GPS;
7. Cálculo de distância, tempo e velocidade média;
8. Histórico e estatísticas;
9. Rotas e descoberta;
10. Compartilhamento de percursos;
11. Contatos de emergência e alertas;
12. Swagger, logs, tratamento global de erros e testes.

A separação em camadas deverá ser aplicada desde o primeiro módulo, evitando a criação de atalhos que dificultem a manutenção posteriormente.

18. Possível evolução futura

A arquitetura foi escolhida para atender ao lançamento inicial, mas deverá permitir evolução gradual.

Curto prazo

- melhoria dos cálculos de desempenho;

- filtros avançados no histórico;
- notificações push;
- favoritos de rotas;
- modo offline com sincronização posterior;
- aprimoramento do painel administrativo.

Médio prazo

- integração com relógios e dispositivos esportivos;
- processamento assíncrono de grandes lotes de pontos GPS;
- cache para consultas frequentes;
- armazenamento especializado para dados geográficos;
- separação de workers para estatísticas e notificações.

Longo prazo

Caso o volume de usuários cresça significativamente, os seguintes módulos poderão ser extraídos do monolito:

- serviço de autenticação;
- serviço de rastreamento GPS;
- serviço de estatísticas;
- serviço de rotas;
- serviço de notificações;
- serviço de segurança e alertas.

Essa migração deverá ocorrer somente quando houver necessidade comprovada de escala, autonomia de equipes ou isolamento de carga. A simples adoção de microsserviços não será considerada um objetivo por si só.

19. Conclusão

A escolha do **monolito em camadas** é a alternativa mais adequada para o cenário atual do projeto **Corrida**.

Ela oferece menor custo, desenvolvimento mais rápido, implantação simples e menor complexidade operacional, atendendo às limitações de orçamento, prazo e tamanho da equipe.

Ao mesmo tempo, a separação entre apresentação, controle, negócio e persistência cria uma base organizada para manutenção e crescimento. Dessa forma, o sistema poderá evoluir gradualmente,

inclusive com a futura extração de módulos para microsserviços, caso o volume de usuários e a complexidade operacional justifiquem essa decisão.

A diretriz principal é entregar uma primeira versão confiável, simples de operar e preparada para crescer sem antecipar complexidades que ainda não são necessárias.

Documento arquitetural do sistema Corrida

Versão 1.0 — 08/09/2026